

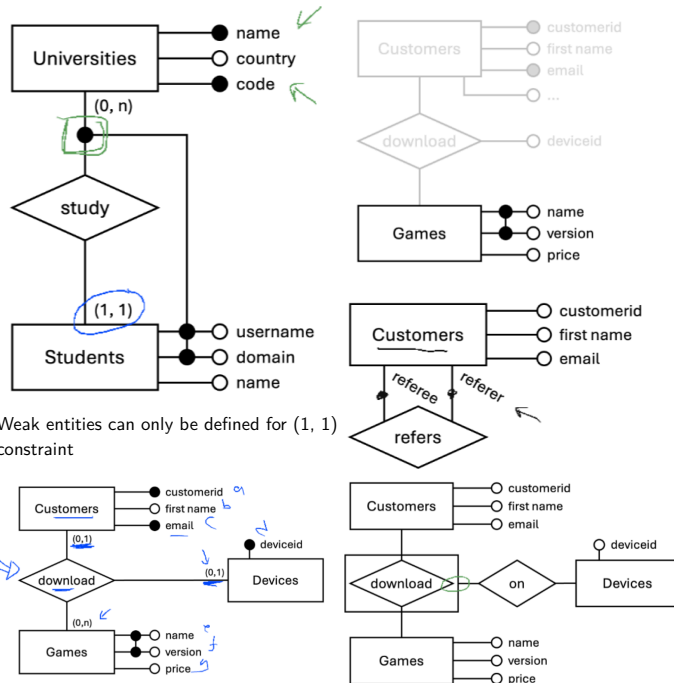
CS4221 Finals Cheatsheet

by Jonathan Loh, River Koh ([GitHub](#))

L02: Conceptual Modelling (ER)

ER to Relational Mapping Rules

- **Strong Entity:** Becomes a table. Attributes become columns. PK remains.
- **Weak Entity:** Table includes its own attributes + PK of the identifying dominant strong entity. PK is the combination of both.
- **1:N Relationship:** Merge relationship into the entity on the 'N' side (add PK of '1' side as Foreign Key).
- **M:N Relationship:** Create a new table. PK is the combination of PKs from participating entities.
- **1:1 Relationship:** Merge into the entity with mandatory participation to minimize NULLs.
- **Nuance - Minimization of NULLs:** Nulls should only exist if they represent missing data meaning, not as an artifact of bad schema translation. Default constraint in SQL should be NOT NULL unless explicitly justified.
- **Derived Attributes:** Dotted line



Weak entities can only be defined for (1, 1) constraint

L03: Data Warehousing & OLAP

OLTP vs OLAP

- **OLTP:** Operational, normalized, short updates, current data. Designed to avoid read-write locks blocking each other. (Typical operational database)
- **OLAP:** Analytical, denormalized, bulk reads/aggregations, historical data (Data warehouses)

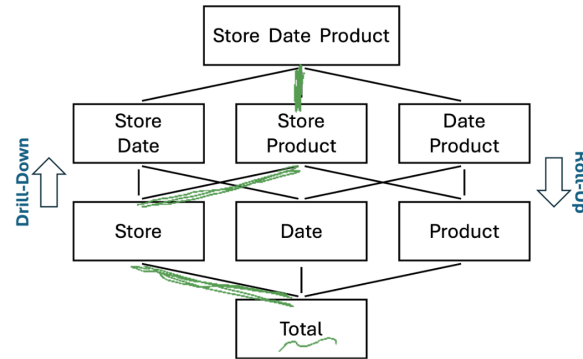
Dimensional Modelling

- **Fact Tables:** Contains business metrics (measures) and foreign keys to dimension tables. Highly massive.

- **Dimension Tables:** Descriptive attributes. Uses surrogate keys (synthetic) rather than natural keys.
- **Star Schema:** Dimension tables are fully denormalized. Fast queries, simpler joins.
- **Snowflake Schema:** Dimension tables are normalized. Saves space, complex joins.

OLAP Operations & SQL Nuances

- **Slice/Dice:** Filtering subsets of a data cube using WHERE.
- **SQL CUBE(A, B):** Generates 2^n grouping sets: (A, B), (A), (B), ().
- **SELECT** product, region, SUM(qty) FROM sales GROUP BY CUBE (product, region)
- **SQL ROLLUP(A, B):** Generates hierarchical sets: (A, B), (A), ().
- e.g. **SELECT** product, region, SUM(qty) FROM sales GROUP BY ROLLUP (product, region)
- In ROLLUP/CUBE output, aggregated columns for subtotals will appear as **NULL**.
- **Drill-Down:** Implemented using GROUP BY



L04: Relational Algebra

Basic Operations

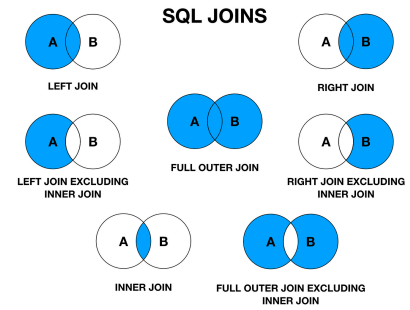
- **Selection (σ_c):** Filters rows based on condition c .
- **Projection (π_L):** Extracts specific columns L . Removes duplicates automatically in relational algebra (pure sets).
- **Cross Product ($\times$):** Cartesian product. $R \times S$ has $n + m$ columns and $n \times m$ rows.
- **Rename Attr:** $\rho_{b \leftarrow a}$
- **Rename Relation:** $\rho(R, S)$

Set Operations (Requires union-compatibility)

- **UNION ($\cup$):** Combines tuples, removing duplicates.
- **INTERSECT ($\cap$):** Tuples present in both R and S .
- **Minus / Difference EXCEPT ($\setminus$):** $R \setminus S$ returns tuples in R but not in S .

Join Operations

- **Theta Join ($\bowtie_\theta$):** $R \bowtie_\theta S \equiv \sigma_\theta(R \times S)$.
- **Natural Join ($\bowtie$):** Equi-join on all matching column names, keeping only one copy of matching columns.
- **Left Outer Join ($\ltimes$):** Keeps all rows from the left table. Unmatched right columns become NULL.
- **Right Outer Join ($\rtimes$):** Keeps all rows from right table.
- **Full Outer Join ($\ltimes$):** Keeps all rows from both tables, filling NULLs where there are no matches.
- **Semi-Join ($\ltimes$):** Returns rows in R that have a match in S . (Optimized for EXISTS).
- **Anti-Join ($\not\bowtie$):** Returns rows in R that DO NOT have a match in S . (Optimized for NOT EXISTS).



Equivalence & Optimization

- **Selection Pushdown:** $\sigma_c(R \bowtie S) \equiv \sigma_c(R) \bowtie S$ (if condition c only applies to R). Reduces rows early!
- **Projection Distribution:** $\pi_L(E_1 \cup E_2) \equiv \pi_L(E_1) \cup \pi_L(E_2)$.

L05: SQL Tuning I (Indexes)

EXPLAIN & Query Plans

- **EXPLAIN:** Est. startup..total cost, rows, width in Bytes

Query Plan
Aggregate (cost=849.87..849.88 rows=1 width=8)
-> Nested Loop (cost=11.33..849.65 rows=89 width=2)
-> Hash Join (cost=11.05..822.10 rows=93 width=6)
Hash Cond: (s.i_id = i.i_id)
-> Seq Scan on stocks s (cost=0.00..692.12 rows=44912 width=10)
-> Hash (cost=11.04..11.04 rows=1 width=4)
-> Seq Scan on items i (cost=0.00..11.04 rows=1 width=4)
Filter: ((i.name)::text = 'Aspirin')::text
-> Index Scan using warehouses_pkey on warehouses w (cost=0.28..0.30 rows=1 width=4)
Index Cond: (w.id = s.w_id)
Filter: ((w.country)::text = 'Indonesia')::text

- **EXPLAIN (ANALYZE):** Actual execution times, rows, loops, filtered rows for each filter

Query Plan
Aggregate (cost=849.87..849.88 rows=1 width=8) (actual time=4.122..4.125 rows=1 loops=1)
-> Nested Loop (cost=11.33..849.65 rows=89 width=2) (actual time=0.138..4.113 rows=72 loops=1)
-> Hash Join (cost=11.05..822.10 rows=93 width=6) (actual time=0.079..3.978 rows=76 loops=1)
Hash Cond: (s.i_id = i.i_id)
-> Seq Scan on stocks s (cost=0.00..692.12 rows=44912 width=10)
-> Hash (cost=11.04..11.04 rows=1 width=4) (actual time=0.033..0.034 rows=1 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 9kB
-> Seq Scan on items i (cost=0.00..11.04 rows=1 width=4)
Filter: ((i.name)::text = 'Aspirin')::text
-> Index Scan using warehouses_pkey on warehouses w (cost=0.28..0.30 rows=1 width=4)
Index Cond: (w.id = s.w_id)
Filter: ((w.country)::text = 'Indonesia')::text
Planning Time: 2.326 ms
Execution Time: 4.156 ms

- **Temporary Table :** CREATE TEMPORARY TABLE stocks_agg AS SELECT * FROM stocks WHERE ...
 - No indexes, no implicit statistics (need ANALYZE temp_t)

Catalogue and Statistics

- **pg_tables:** name, index, triggers in DB
- **pg_attribute:** table columns, one row for every column
- **pg_statistics:** created by ANALYZE, approximate statistics about contents in DB
- **pg_stats:** user-readable version of pg_statistics

Scan Types

- **Sequential Scan:** Reads every page.
- **Index Scan:** Traverses B-Tree, then fetches rows from the heap. Causes random I/O.
- **Index-Only Scan:** Finds all requested columns directly in the index. No heap fetch!
- **Covering Index:** CREATE INDEX idx ON T(A) INCLUDE (B)
- **Predicate Index:** CREATE INDEX idx ON T(A) WHERE A > 600

L06: SQL Tuning II & Maintenance

Join Implementations

- **Nested Loop Join:** Outer table scanned once, inner table scanned for every outer row. Good if inner table is indexed.
- **Hash Join:** Builds a hash table on the inner relation. Favored for large unsorted tables. Requires work_mem.
- **Merge Join:** Sorts both inputs, then merges. Best if inputs are already sorted by B-Tree indexes.

Table Maintenance

- **VACUUM:** Reclaims space from dead tuples (MVCC artifacts).
- **VACUUM FULL:** Rewrites table to disk, shrinking file. Takes an exclusive lock.
- **CLUSTER:** Rewrites table so physical order matches an index.

Common SQL Nuances & Traps

DROP vs DELETE vs TRUNCATE

- **DELETE (DML):** Removes rows row-by-row. Logs each deletion. Slower, but triggers fire. Can use WHERE.
- **TRUNCATE (DDL-like DML):** Deletes all rows instantly by deallocating pages. No triggers fired. Resets IDENTITY/SERIAL columns. Cannot use WHERE.
- **DROP TABLE (DDL):** Completely removes the table data, structure, indexes, and privileges.

EXCEPT vs NOT IN vs NOT EXISTS

- **EXCEPT:** Set operation, automatically removes duplicates.
- **NOT IN: Danger:** Fails completely (returns 0 rows) if the subquery returns ANY NULL values.
- **NOT EXISTS:** Safer, usually optimized into an Anti-Join.

Other Traps

- **NULL Comparisons:** x = NULL evaluates to UNKNOWN (treated as false in WHERE). Always use x IS NULL.
- **WHERE vs HAVING:** WHERE filters rows **before** aggregation. HAVING filters groups **after** aggregation.
- **UNION vs UNION ALL:** UNION sorts and removes duplicates (expensive). Use UNION ALL if duplicates are acceptable or impossible.
- **Execution Order:** FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY. (You cannot use an alias defined in SELECT within the WHERE clause!).

Cross-only Join Cheatsheet (RA)

Core Trick

- **Theta Join:** $R \bowtie_{\theta} S \equiv \sigma_{\theta}(R \times S)$
- **Equijoin:** $R \bowtie_{R.a=S.b} S \equiv \sigma_{R.a=S.b}(R \times S)$

Natural Join (only using ×, σ, π, ρ)

- Let $R(A, B)$ and $S(B, C)$.
- Rename to avoid duplicate attribute names: $S' := \rho_{B' \leftarrow B}(S)$
 - Cross + select on equality: $T := \sigma_{R.B=S'.B'}(R \times S')$
 - Drop duplicate join column: $R \Join S \equiv \pi_{A,B,C}(T)$

Self-Join (needs rename)

- $\sigma_{\theta}(\rho_{R1}(R) \times \rho_{R2}(R))$

Semi-Join (EXISTS / ANY vs IN) from cross join

- Definition idea: join then keep only left attr
- $R \ltimes_{\theta} S \equiv \delta(\pi_{(R)}(\sigma_{\theta}(R \times S)))$

Anti-Join (NOT EXISTS / LEFT JOIN) from cross join

- $R \rhd_{\theta} S \equiv R \setminus (R \ltimes_{\theta} S)$

Assignment 1 identities

- $R \Join_{[\text{cond}]} S \equiv \delta\left(\sigma_{[\text{cond}]}(R \times S)\right) \cup \left(R \rhd_{[\text{cond}]} S\right)$
- $R \div S \equiv \pi_{[R-S]}(R) \setminus \left(\pi_{[R-S]}\left(\pi_{[R]}\left(\pi_{[R-S]}(R) \times S\right) \setminus R\right)\right)$

Functional Dependencies (FDs)

Definition: $X \rightarrow Y$ holds if tuples agreeing on X must also agree on Y

- **Trivial:** $Y \subseteq X$ (Always true inherently)
- **Non-trivial:** $Y \not\subseteq X$
- **Completely Non-trivial:** $X \cap Y = \emptyset$

Keys & Attributes

- **Superkey:** Set of attributes S such that $S \rightarrow R$ (determines the entire tuple).
- **Candidate Key (CK):** Minimal superkey (no proper subset is a superkey).
- **Prime Attribute:** Attribute belonging to **any** candidate key.
- **Non-Prime Attribute:** Attribute not in any CK.

Normal Forms (Presume 1NF)

2NF: No partial dependencies. For every non-trivial $X \rightarrow A$:

- A is prime, OR X is NOT a proper subset of any CK.

(Intuition: **Non-prime attributes must depend on the whole key, not just part of it**).

3NF: No transitive dependencies for non-prime attributes. For every non-trivial $X \rightarrow A$:

- X is a superkey, OR A is prime.

(Intuition: **Attributes depend on the key, the whole key, and nothing but the key**).

BCNF: Every determinant is a superkey. For every non-trivial $X \rightarrow A$:

- X is a superkey.

(Intuition: **Stricter 3NF. Even prime attributes cannot depend on non-superkeys**).

Multivalued Dependencies (MVD)

Intuition: $X \twoheadrightarrow Y$ means for a given X , the set of Y values is completely independent of the rest of the attributes $(R - X - Y)$.

- **Definition:** If tuples (x, y_1, z_1) and (x, y_2, z_2) exist, then the swapped tuples (x, y_1, z_2) and (x, y_2, z_1) must also exist.
- **Trivial MVDs:** $Y \subseteq X$ or $X \cup Y = R$.

Covers

- **Minimal Cover:** A set of FDs where, LHS minimal, RHS minimal, FD set minimal
- **Compact Cover:** A set of FDs where no two FDs share the same LHS
- **Canonical Cover:** A set of FDs that is BOTH **Compact** and **Minimal**

Algorithms

1. Attribute Closure (S^+) **Input:** Set S , FDs Σ . **Output:** All attributes A such that $S \rightarrow A$.

1. Start with result $\Gamma = S$.
2. While there's an FD $X \rightarrow Y$ in Σ where $X \subseteq \Gamma$: Add Y to Γ .
3. Repeat until Γ stops changing.

2. Minimal Cover (F_c)

1. **RHS Singletons:** Split RHS of all FDs (e.g., $X \rightarrow YZ \Rightarrow X \rightarrow Y, X \rightarrow Z$).
2. **LHS Reduction:** For each $X \rightarrow A$, if removing an attribute B from X still allows $X - \{B\} \rightarrow A$ to be derived using the **current** set of FDs, replace $X \rightarrow A$ with $X - \{B\} \rightarrow A$.
3. **Remove Redundancies:** For each FD $X \rightarrow A$, if $X \rightarrow A$ can be derived from the remaining FDs (check if $A \in X^+$ using all **other** FDs), delete $X \rightarrow A$.

3. 3NF Synthesis (Bernstein's) Guarantees: Lossless Join + Dependency Preservation.

1. Find Minimal Cover F_c of Σ .
2. Combine FDs with the same LHS: $X \rightarrow A$ and $X \rightarrow B \Rightarrow X \rightarrow AB$.
3. For each combined FD $X \rightarrow Y$ in F_c , create a relation $R_i = X \cup Y$.
4. **Subsumption:** Delete relations subsumed by others (if $R_i \subseteq R_j$, drop R_i).

5. **CK Check:** If no relation contains a Candidate Key of the original R , add one relation containing any CK.

4. BCNF Decomposition Guarantees: Lossless Join (but may lose Dependency Preservation).

1. Find a violation $X \rightarrow A$ (non-trivial, X is not a superkey).
2. Decompose R into:
 - $R_1 = X^+$ (The closure of the violator)
 - $R_2 = (R - X^+) \cup X$ (Everything else + the violator LHS)
3. Repeat recursively for R_1, R_2 until all are in BCNF.

5. The Chase Algorithm

The Core Engine (Rules)

- **FD Rule (Value-Generating):** If $X \rightarrow Y$ and two rows agree on X , force them to agree on Y . Upgrade b to a (prioritize distinguished). If both are b , match the higher-subscripted b to the lower one.
- **MVD Rule (Tuple-Generating):** If $X \twoheadrightarrow Y$ and two rows agree on X , generate new rows. Swap the Y values of the two matching rows while keeping the remaining attributes $(R - X - Y)$ the same. Add these newly generated rows to the bottom of the table.

App 1: Testing Lossless Join (Join Dependencies)

- **Goal:** Test if a decomposition into n fragments is lossless.
- **Setup:** Create a table with n rows. Fill cell (i, j) with distinguished variable a_j if attribute A_j is in fragment R_i . Otherwise, fill with non-distinguished $b_{\{i,j\}}$.
- **Action:** Apply FD and MVD rules repeatedly until a fixed point is reached.
- **Termination:** It is lossless if a row of **all** a 's is eventually generated. Otherwise, it is lossy, and the final table serves as a counterexample.

App 2: Testing Implication ($\Sigma \models X \rightarrow Y$ or $X \twoheadrightarrow Y$)

- **Goal:** Test if a specific dependency is logically entailed by a set of dependencies.
- **Setup for FD ($X \rightarrow Y$):** Create a table with 2 rows. Distinguish (a) all attributes in X for both rows. Fill the rest with distinct b 's.
- **Setup for MVD ($X \twoheadrightarrow Y$):** Create a table with 2 rows. Row 1: Distinguish (a) all attributes in $X \cup Y$. Row 2: Distinguish (a) all attributes in $X \cup (R - X - Y)$. Fill the rest with distinct b 's.
- **Termination:** Apply rules. For FDs, it is implied if the two rows eventually agree on all Y attributes. For MVDs, it is implied if **any** row becomes entirely distinguished (all a 's).

Axioms & Rules

Armstrong's Axioms for FDs:

- **Reflexivity:** $Y \subseteq X \Rightarrow X \rightarrow Y$
- **Augmentation:** $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
- **Transitivity:** $X \rightarrow Y$ and $Y \rightarrow Z \Rightarrow X \rightarrow Z$
- **Union:** $X \rightarrow Y$ and $X \rightarrow Z \Rightarrow X \rightarrow YZ$
- **Decomposition:** $X \rightarrow YZ \Rightarrow X \rightarrow Y$ and $X \rightarrow Z$
- **Pseudotransitivity:** $X \rightarrow Y$ and $WY \rightarrow Z \Rightarrow WX \rightarrow Z$.

MVD Axioms (Fagin's Rules):

- **Complementation:** $X \twoheadrightarrow Y \Rightarrow X \twoheadrightarrow (R - X - Y)$ (If **Y** is independent, the rest is too).
- **Augmentation:** $X \twoheadrightarrow Y$ and $V \subseteq W \Rightarrow XW \twoheadrightarrow YV$ (Allows padding the left with a superset W and the right with a subset V).
- **Transitivity:** $X \twoheadrightarrow Y$ and $Y \twoheadrightarrow Z \Rightarrow X \twoheadrightarrow (Z - Y)$
- **Replication:** $X \rightarrow Y \Rightarrow X \twoheadrightarrow Y$ (Every FD is inherently a stricter MVD).
- **Multivalued Union:** $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z \Rightarrow X \twoheadrightarrow YZ$.
- **Multivalued Intersection:** $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z \Rightarrow X \twoheadrightarrow Y \cap Z$.
- **Exact Difference:** $X \twoheadrightarrow Y$ and $X \twoheadrightarrow Z \Rightarrow X \twoheadrightarrow Y - Z$.
- **Coalescence:** If $X \twoheadrightarrow Y$ and there exists W such that $W \cap Y = \emptyset$ and $W \rightarrow Z$ where $Z \subseteq Y$, then $X \rightarrow Z$.